ASSIGNMENT - 16

Thumma Hasini | 2303A52076 | Batch - 37

Title: Database Design, Query Optimization, and Analysis using AI Tools

Task 1 – Hospital Management Database

Prompt Used

Create schema and queries for a Hospital Management System with tables Doctors, Patients, and Appointments including constraints and analytical queries.

Code

```sql
# Generate a normalized SQL schema for a Hospital Management System

CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialization TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT NOT NULL UNIQUE,
    hire_date DATE NOT NULL,
    CHECK (hire_date <= DATE('now'))
);

CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth DATE NOT NULL,
    gender TEXT CHECK (gender IN ('M','F','O')),
    email TEXT UNIQUE,
    phone TEXT UNIQUE,
    address TEXT,
    CHECK (date_of_birth <= DATE('now'))
);

CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATETIME NOT NULL,
    reason TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'Scheduled',
    CHECK (status IN ('Scheduled', 'Completed', 'Cancelled', 'No-sh
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF

lab1.sql

```sql
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATETIME NOT NULL,
    reason TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'Scheduled',
    CHECK (status IN ('Scheduled', 'Completed', 'Cancelled', 'No-sh
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

CREATE UNIQUE INDEX ux_appointments
ON Appointments (doctor_id, patient_id, appointment_date);



INSERT INTO Doctors (first_name, last_name, specialization, email, |
VALUES
('John', 'Doe', 'Cardiology', 'john@gmail.com', '1234567890', DATE(
('Jane', 'Smith', 'Neurology', 'jane@gmail.com', '9876543210', DATE

INSERT INTO Patients (first_name, last_name, date_of_birth, gender)
VALUES
('Alice', 'Brown', '2000-05-10', 'F'),
('Bob', 'White', '1998-03-15', 'M');

INSERT INTO Appointments (doctor_id, patient_id, appointment_date, |
VALUES
(1, 1, DATETIME('now'), 'General Checkup', 'Completed'),
(1, 2, DATETIME('now'), 'Heart Issue', 'Scheduled'),
(2, 1, DATETIME('now'), 'Headache', 'Completed');
```

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 4 of 4

name
sqlite_sequence
Doctors
Patients
Appointments

SQL ▼ 1 / 1 1 - 2 of 2

appointment_id	appointment_date	reason	status	patient_name
1	2026-03-18 07:59:09	General Checkup	Completed	Alice
2	2026-03-18 07:59:09	Heart Issue	Scheduled	Bob

SQL ▼ 1 / 1 1 - 2 of 2

appointment_id	appointment_date	reason	status	doctor_name
1	2026-03-18 07:59:09	General Checkup	Completed	John
3	2026-03-18 07:59:09	Headache	Completed	Jane

SQL ▼ 1 / 1 1 - 2 of 2

first_name	total_patients
Jane	1
John	1

Ln 92, Col 15 Spaces: 4 UTF-8 CRLF { } SQLite SQLite: lab.db Go Live

Explanation

This task focuses on designing a Hospital Management System database.

- The Doctors table stores doctor details.
- The Patients table maintains patient records.
- The Appointments table links doctors and patients.

Foreign keys ensure relationships:

- A doctor can have multiple appointments.
- A patient can book multiple appointments.

Queries were created to:

- List all appointments for a specific doctor
- Retrieve patient history
- Count total patients treated by each doctor

This schema follows normalization principles and ensures efficient data handling.

Task 2 – Online Shopping Database

Prompt Used

Design an e-commerce database with Users, Products, Orders, and OrderDetails tables, including analytical queries and optimization suggestions.

Code

```sql
# Design a relational database schema for an e-commerce system with ta
-- Table: Users
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (email LIKE '%_@__%.__%')
);

-- Table: Products
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    sku TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
    description TEXT,
    price REAL NOT NULL CHECK (price >= 0),
    available_quantity INTEGER NOT NULL CHECK (available_quantity >= 0),
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Table: Orders
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status TEXT NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (user_id)
        REFERENCES Users(user_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CHECK (status IN ('Pending','Paid','Shipped','Completed','Cancelled'))
);
```

C:\Users\lenovo\Desktop\lab.db\lab2.sql

```sql
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price REAL NOT NULL CHECK (unit_price >= 0),
    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (product_id)
        REFERENCES Products(product_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    UNIQUE (order_id, product_id)
);

-- 1) Retrieve all orders by a user
SELECT
    o.order_id,
    o.order_date,
    o.status,
    od.order_detail_id,
    od.product_id,
    p.name AS product_name,
    od.quantity,
    od.unit_price,
    od.quantity * od.unit_price AS line_total
FROM Orders o
LEFT JOIN OrderDetails od ON o.order_id = od.order_id
LEFT JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1
ORDER BY o.order_date DESC, o.order_id;

-- 2) Find the most purchased product
SELECT
```

Ln 19, Col 24 Spaces: 4 UTF-8 CRLF { } SQLite

lab2.sql

```sql
SELECT
    p.product_id,
    p.name,
    SUM(od.quantity) AS total_quantity_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY total_quantity_sold DESC
LIMIT 1;

-- 3) Calculate total revenue in a given month
SELECT
    '2026' AS Year,
    '03' AS Month,
    SUM(od.quantity * od.unit_price) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y', o.order_date) = '2026'
  AND strftime('%m', o.order_date) = '03';

-- Index suggestions
CREATE INDEX idx_orders_user_id ON Orders (user_id, order_date);
CREATE INDEX idx_orderdetails_order_id ON OrderDetails (order_id, product_id);
CREATE INDEX idx_orderdetails_product_id ON OrderDetails (product_id);
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 1 of 1

Year	Month	total_revenue
2026	03	NULL

Explanation

This task implements a real-world online shopping system.

- Users table stores customer data.
- Products table contains product information.
- Orders table tracks purchases.
- OrderDetails stores items per order.

Queries include:

- Retrieving all orders by a user
- Finding the most purchased product
- Calculating monthly revenue

AI suggested improvements:

- Splitting categories into separate tables
- Using indexes for faster queries
- Avoiding redundancy through normalization

Task 3 – Library Management System

Prompt Used

Design a library system with Books, Members, and Loans tables including indexing strategies and queries.

Code

```sql
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    isbn TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    publisher TEXT,
    published_year INTEGER,
    total_copies INTEGER NOT NULL CHECK (total_copies >= 0),
    available_copies INTEGER NOT NULL CHECK (available_copies >= 0)
);

-- Table: Members
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    joined_date DATE NOT NULL DEFAULT DATE('now')
);

-- Table: Loans
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date DATE NOT NULL DEFAULT DATE('now'),
    return_date DATE NULL,
    due_date DATE,
    status TEXT NOT NULL DEFAULT 'Loaned',
    CHECK (status IN ('Loaned','Returned','Lost','Overdue')),
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
```

Ln 105, Col 15 Spaces: 4 UTF-8 CRLF {} SQLite

```sql
SELECT
    b.book_id,
    b.title,
    b.author,
    m.member_id,
    m.first_name,
    m.last_name,
    l.loan_date,
    l.due_date,
    l.status
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
ORDER BY l.loan_date DESC;

-- Query 2: Find overdue books (loan > 30 days and not returned)
SELECT
    l.loan_id,
    b.book_id,
    b.title,
    m.member_id,
    m.first_name,
    m.last_name,
    l.loan_date,
    l.due_date,
    CAST((julianday('now') - julianday(l.loan_date)) AS INTEGER) AS days_on_l
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
  AND (julianday('now') - julianday(l.loan_date)) > 30
ORDER BY days_on_loan DESC;
```

```sql
WHERE l.return_date IS NULL
ORDER BY days_on_loan DESC;

-- Query 3: Count books loaned by each member
SELECT
    m.member_id,
    m.first_name,
    m.last_name,
    COUNT(l.loan_id) AS total_loans,
    SUM(CASE WHEN l.return_date IS NULL THEN 1 ELSE 0 END) AS active_loans
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.first_name, m.last_name
ORDER BY total_loans DESC;

-- Indexing strategies
CREATE INDEX idx_loans_book_member ON Loans (book_id, member_id);
CREATE INDEX idx_loans_return_loan_date ON Loans (return_date, loan_date);
CREATE INDEX idx_loans_loan_date ON Loans (loan_date);
CREATE UNIQUE INDEX idx_members_email ON Members (email);
CREATE UNIQUE INDEX idx_books_isbn ON Books (isbn);

INSERT INTO Books (isbn, title, author, total_copies, available_copies)
VALUES ('123', 'DBMS Book', 'Author A', 5, 5);

INSERT INTO Members (first_name, last_name, email)
VALUES ('John', 'Doe', 'john@gmail.com');

INSERT INTO Loans (book_id, member_id)
VALUES (1, 1);
```

Explanation

This task models a Library Management System.

- Books table tracks book inventory
- Members table stores user details
- Loans table manages borrow/return activity

Queries implemented:

- Retrieve issued books

- Find overdue books
- Count loans per member

Indexing improves performance:

- Faster searches on loan_date
- Efficient joins using foreign keys
- Quick member lookup via email index

Task 4 – Query Optimization

Prompt Used

Analyze and optimize inefficient SQL queries using joins instead of subqueries.

Code

lab4.sql

```sql
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;

-- Table: Authors
CREATE TABLE Authors (
    author_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    country TEXT NOT NULL
);

-- Table: Books
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(
);

-- Insert sample data
INSERT INTO Authors (name, country)
VALUES
('J.K. Rowling', 'UK'),
('George Orwell', 'UK'),
('Mark Twain', 'USA');

INSERT INTO Books (title, author_id)
VALUES
('Harry Potter', 1),
('1984', 2),
('Animal Farm', 2),
('Tom Sawyer', 3);

-- Query: Books written by UK authors
SELECT b.*
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 3 of 3

book_id	title	author_id
1	Harry Potter	1
2	1984	2
3	Animal Farm	2

Explanation

This task demonstrates query optimization techniques.

- Original query uses a subquery
- Optimized query uses JOIN

Key observations:

- JOIN improves performance
- Reduces execution time for large datasets
- Both queries produce identical results

This highlights the importance of writing efficient SQL queries.

Task 5 – University Course Registration System

Prompt Used

Design a course registration system with Students, Courses, Faculty, and Registrations tables along with analytical queries.

Code

lab5.sql

```sql
SELECT
    s.student_id,
    s.first_name,
    s.last_name,
    s.email,
    r.registration_date,
    r.status
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
WHERE r.course_id = 1   -- use ? instead of @Co
ORDER BY s.last_name, s.first_name;

-- Query 2
SELECT
    f.faculty_id,
    f.first_name,
    f.last_name,
    f.department,
    COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2
ORDER BY course_count DESC;

-- Query 3
SELECT
    c.course_id,
    c.course_code,
    c.title,
    f.first_name AS faculty_first,
    f.last_name AS faculty_last,
    COUNT(r.registration_id) AS registratic
```

SQL ▾ 1 / 1 1 - 2 of 2

student_id	first_name	last_name	email	reg
2	Bob	Brown	bob@mail.com	202
1	Alice	Smith	alice@mail.com	202

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 1 of 1

course_id	course_code	title	faculty_first	fa
1	CS101	Intro to CS	John	Do

Ln 71, Col 11 Spaces: 4 UTF-8 CRLF { } SQLite SQLite: test.db Go Live

lab5.sql

```sql
-- Table: Students
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCR
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    enrollment_date DATE NOT NULL DEFAULT (
);

-- Table: Faculty
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY AUTOINCREMENT
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    department TEXT NOT NULL
);

-- Table: Courses
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_code TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    credits INTEGER NOT NULL CHECK (credits > 0)
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id)
        REFERENCES Faculty(faculty_id) ON DELETE
);

-- Table: Registrations
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCR
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL
```

lab5.sql

```sql
LEFT JOIN Registrations r ON c.course_id =
LEFT JOIN Faculty f ON c.faculty_id = f.fac
GROUP BY c.course_id
ORDER BY registration_count DESC
LIMIT 10;

-- Indexes
CREATE INDEX idx_courses_faculty_id ON Cour
CREATE INDEX idx_registrations_student_id C
CREATE INDEX idx_registrations_course_id ON
SELECT name FROM sqlite_master WHERE type='
-- Insert Faculty
INSERT INTO Faculty (first_name, last_name,
VALUES ('John', 'Doe', 'john@uni.edu', 'CSE');

-- Insert Students
INSERT INTO Students (first_name, last_name,
VALUES ('Alice', 'Smith', 'alice@mail.com'),
       ('Bob', 'Brown', 'bob@mail.com');
       ('Nina', 'Davis', 'nina@mail.com');
-- Insert Courses
INSERT INTO Courses (course_code, title, cr
VALUES ('CS101', 'Intro to CS', 3, 1);

-- Insert Registrations
INSERT INTO Registrations (student_id, cour
VALUES (1, 1, 'Active'),
       (2, 1, 'Active');
       (3, 3, 'Active');

SELECT * FROM Students;
```

Explanation

This task builds a university course registration system.

- Students table stores student data
- Faculty table contains instructor details
- Courses table links faculty to courses
- Registrations table manages student enrollments

Relationships:

- One student → many courses
- One faculty → many courses

Queries include:

- Listing students in a course
- Finding faculty teaching more than 2 courses
- Identifying most popular courses

This design ensures scalability and efficient querying.

Final Conclusion: Role of AI in AIAC Lab

AI-assisted coding significantly enhanced the efficiency and learning experience throughout this assignment.

- AI helped generate structured database schemas quickly
- Reduced syntax errors and debugging time
- Provided optimization strategies for better performance
- Enabled understanding of real-world database design concepts

AI tools like Copilot act as intelligent assistants, guiding developers in writing clean and efficient code. However, understanding the underlying logic remains essential to effectively use these tools.

Overall, AIAC demonstrates how AI can accelerate development while also improving conceptual clarity in database management and query optimization.